Solution: Remove Duplicates from Sorted List (easy)

Problem Statement:

Given a sorted linked list, remove all the duplicate elements to leave only distinct numbers. The linked list should remain sorted, and the modified list should be returned.

Examples

Example 1:

- **Input:** 1 -> 1 -> 2
- **Output:** 1 -> 2
- **Justification:** Since 1 is repeated, we remove the duplicate to leave a sorted list of unique numbers.

Example 2:

- **Input:** 1 -> 2 -> 2 -> 3
- **Output:** 1 -> 2 -> 3
- **Justification:** Here, 2 is the duplicate element, and by removing it, we obtain a list containing only distinct elements.

Example 3:

- **Input:** 3 -> 3 -> 3
- **Output:** 3
- **Justification:** We remove the repeated 3s to leave a single 3 in the list.

Solution

To solve this problem, start at the beginning of the list. If the list is empty or has only one item, return the list. If not, look at each element in the list one by one. If two elements next to each other that are the same, skip over the second one. Do this by changing the link of the first element to the element after the second one. Keep doing this until you reach the end of the list. This way, all the repeated elements are removed, and you still keep the list in order.

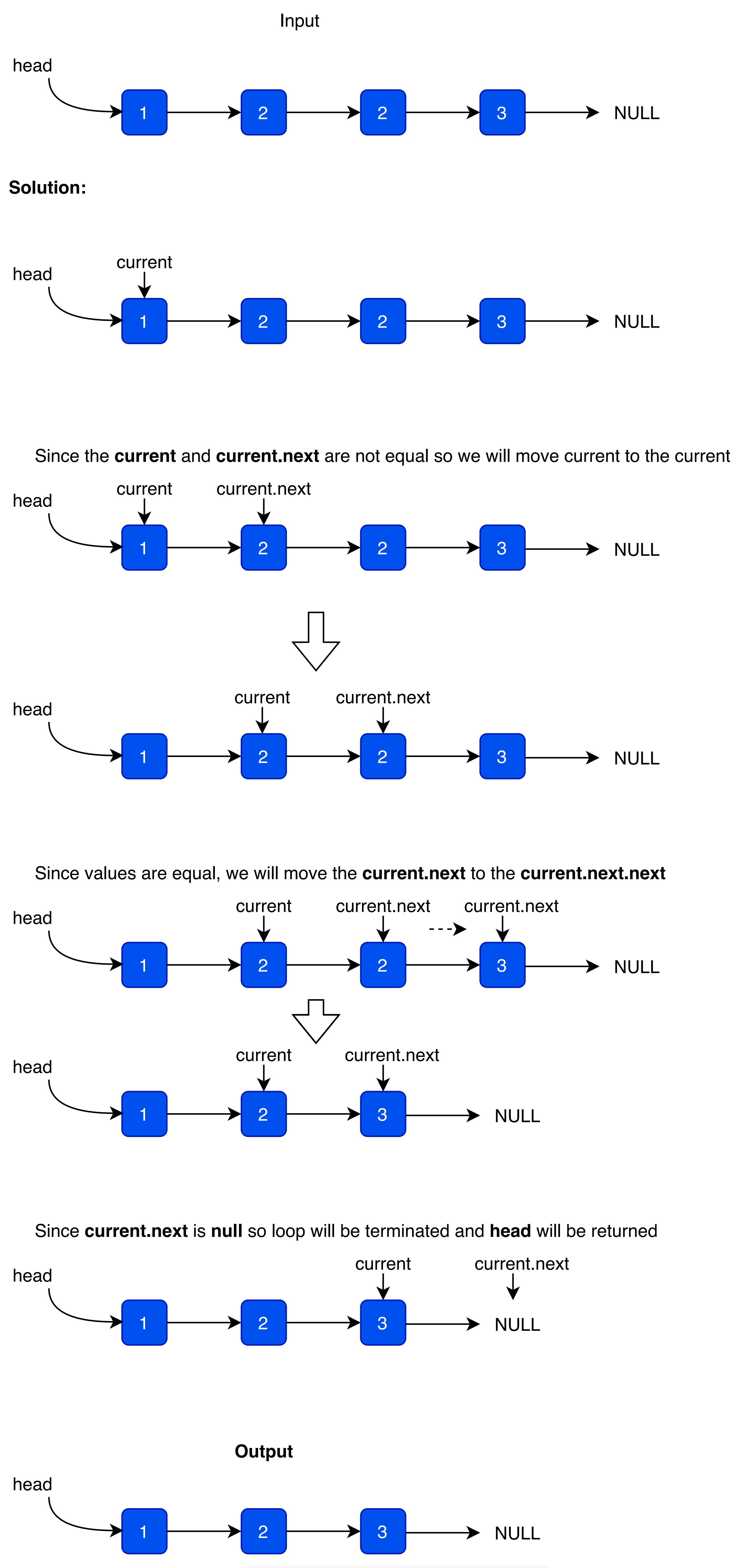
Here is the step-by-step solution.

1. **Start at the Head**: Begin at the head of the linked list.
2. **Check for Empty or Single-Node List:**
   - If the list is empty or contains only one node, return the list as is, since there are no duplicates to remove.
3. **Traversal:**
   - Initialize a pointer, say `current`, to the head of the list.
   - While `current` and `current.next` are not null (ensuring there are at least two nodes to compare):
     - Check if `current`'s value is equal to `current.next`'s value.
       - If they are equal, it indicates a duplicate.
         - Update `current.next` to `current.next.next`. This skips over the duplicate node and effectively removes it from the list.
       - If they are not equal, move `current` to the next node (`current = current.next`). This step is crucial to progress through the list.
4. **Final List:**
   - Once the end of the list is reached (i.e., `current` or `current.next` becomes null), return the head of the modified list.

Algorithm Walkthrough:

- Start at the head of the linked list.
- For each node:
  - Compare the current node's value with its successor.
  - If they are identical, remove the duplicate by updating the next pointer of the current node to its successor's successor.
  - If they are distinct, move to the next node.
- Continue until the end of the list is reached.

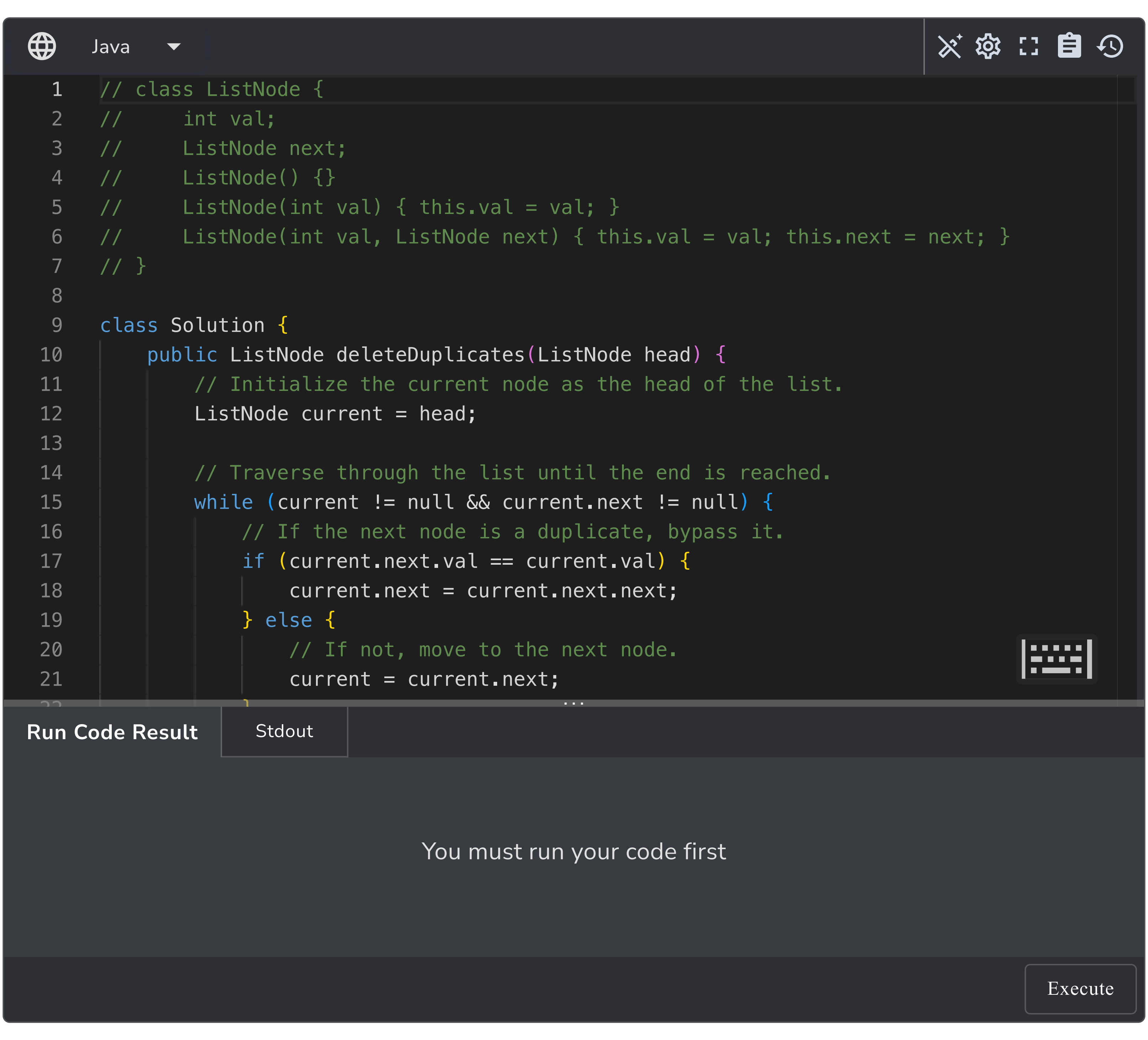
Here is the visual representation of the algorithm:


Input
head
1
2
2
3
NULL
Solution:
head
current
1
2
2
3
NULL
Since the current and current.next are not equal so we will move current to the current.next
head
current
current.next
1
2
2
3
NULL
head
current
current.next
1
2
2
3
NULL
Since values are equal, we will move the current.next to the current.next.next
head
current
current.next
current.next
1
2
2
3
NULL
head
current
current.next
1
2
3
NULL
Since current.next is null so loop will be terminated and head will be returned
head
current
current.next
1
2
3
NULL
Output
head
1
2
3
NULL

Remove Duplicates from Sorted List

Code

Here is the code for this algorithm:



```java
// class ListNode {
//     int val;
//     ListNode next;
//     ListNode() {}
//     ListNode(int val) { this.val = val; }
//     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
// }

class Solution {
    public ListNode deleteDuplicates(ListNode head) {
        // Initialize the current node as the head of the list.
        ListNode current = head;

        // Traverse through the list until the end is reached.
        while (current != null && current.next != null) {
            // If the next node is a duplicate, bypass it.
            if (current.next.val == current.val) {
                current.next = current.next.next;
            } else {
                // If not, move to the next node.
                current = current.next;
```

You must run your code first

1. Time Complexity:

The time complexity of this algorithm is determined by the number of elements in the linked list since we are traversing the list once.

- We traverse each node of the linked list exactly once.
- For each node, we perform constant-time operations to check whether the next node is a duplicate and possibly update the `next` pointer.

Therefore, the time complexity of this algorithm is **O(n)**, where **n** is the number of nodes in the linked list.

2. Space Complexity:

The space complexity of an algorithm analyzes the amount of memory space required relative to the input size.

- The algorithm uses a constant amount of space to store pointers (like the `current` pointer) and temporary variables, regardless of the input size.
- We are not using any additional data structures that scale with the input size.
- The algorithm modifies the input linked list in place and does not create any new nodes or linked lists.

Therefore, the space complexity of this algorithm is **O(1)**, indicating constant space usage.

← Previous
Problem 2: Remove Duplicates From Sorted List (Easy)
Next →
Problem 3: Merge Two Sorted Lists (Easy)
☐ Mark as Completed